Serviço de Autenticação Web — Documentação Consolidada (APS)

Autor: David Daves Ferreira Pinto
Disciplina: Domínios de Software
Docente Responsável: Profa. Adriana Silveira de Souza
Repositório / Projeto: Documentação Arquitetural - Autenticação Web
Data de Consolidação: 12 de novembro de 2026

Sumário

SEÇÃO 1 - AMBIENTE OPERACIONAL WEB

1.1. Fundamentos do Ambiente Operacional Web
1.2. Arquitetura Cliente-Servidor
1.3. Organização Geral de uma Aplicação Web
1.4. Protocolos e Comunicação no Ambiente Web
1.5. Considerações sobre o Ambiente Operacional
1.6. Componentes Operacionais do Ambiente Web

SEÇÃO 2 - PROJETO DA ARQUITETURA E DESENHO DETALHADO

2.1. Especificação de Requisitos de Software e Direcionadores Arquiteturais
2.2. Descrição Arquitetural e Especificação dos Componentes
2.3. Diagramas Arquiteturais e Comportamentais (C4 e UML)

Referências Bibliográficas
Apêndice I - Registro de Utilização de Inteligência Artificial
Apêndice II - Registro dos Prompts de Inteligência Artificial

---

SEÇÃO 1 — AMBIENTE OPERACIONAL WEB

1.1. Fundamentos do Ambiente Operacional Web

As aplicações Web estão presentes em diversas atividades do cotidiano, permitindo que usuários acessem serviços e informações por meio da Internet. Embora essa interação pareça simples, seu funcionamento envolve diferentes tecnologias e componentes que trabalham de maneira coordenada para receber, processar e responder às solicitações realizadas pelos respectivos usuários.

De modo geral, uma aplicação Web depende da comunicação entre o navegador, os servidores responsáveis pelo processamento e, quando necessário, os sistemas de armazenamento de dados. Cada um desses elementos desempenha uma função específica, contribuindo para que as operações sejam executadas de maneira organizada.

Contexto do Projeto: Este projeto tem como objetivo documentar a arquitetura de um sistema Web de autenticação de usuários. Compreender esse ambiente é fundamental, pois o processo de autenticação envolve não apenas o recebimento das credenciais, mas também sua transmissão, verificação e o retorno de uma resposta ao cliente.

1.2. Arquitetura Cliente-Servidor

A arquitetura cliente-servidor constitui o modelo fundamental para a estruturação e operação de aplicações integradas à Web. Esse modelo organiza as responsabilidades entre o cliente, que solicita recursos ou serviços, e o servidor, que recebe as requisições e realiza o processamento necessário.

- **Cliente:** O componente que inicia a comunicação na ponta do usuário (tipicamente representado por navegadores Web ou aplicações móveis). Sua função primária consiste em formatar e disparar requisições específicas com base nas interações da interface.
- **Servidor:** O nó central da infraestrutura que permanece em estado de escuta ativa. É encarregado de receber as demandas enviadas pelo cliente, processar as regras de negócio lógicas e retornar os recursos ou dados solicitados de forma isolada.

O fluxo operacional dessa arquitetura é regido por um ciclo contínuo de Requisição e Resposta. Esse intercâmbio de dados transaciona essencialmente através do protocolo HTTP (Hypertext Transfer Protocol), ou, obrigatoriamente em sistemas que manipulam dados sensíveis, por meio do HTTPS (Hypertext Transfer Protocol Secure). A variante segura introduz uma camada de criptografia (TLS) diretamente no canal de comunicação, mitigando riscos de interceptação e garantindo a integridade dos pacotes trafegados entre os dois pontos da rede.

```
[ Cliente (Browser) ]  ── Requisição (HTTP/HTTPS) ──►  [ Servidor de
Aplicação ]
[                   ]  ◄── Resposta (HTTP/HTTPS)  ──  [
]
```

1.2.1. Fluxo de Processamento

1. **Solicitação:** O cliente envia uma requisição para acessar um recurso ou executar uma operação.
2. **Recebimento:** O servidor recebe a requisição e identifica o serviço solicitado.

3. **Processamento:** O servidor executa as operações necessárias, podendo consultar outros componentes ou um banco de dados.
4. **Resposta:** O resultado é encaminhado ao cliente, que interpreta as informações recebidas.

Essa organização permite distribuir as responsabilidades e manter uma separação nítida entre a interface utilizada pelo usuário e o processamento que é realizado no servidor (MDN WEB DOCS, s.d.-a).

1.2.2. Aplicação do Modelo na Autenticação de Usuários

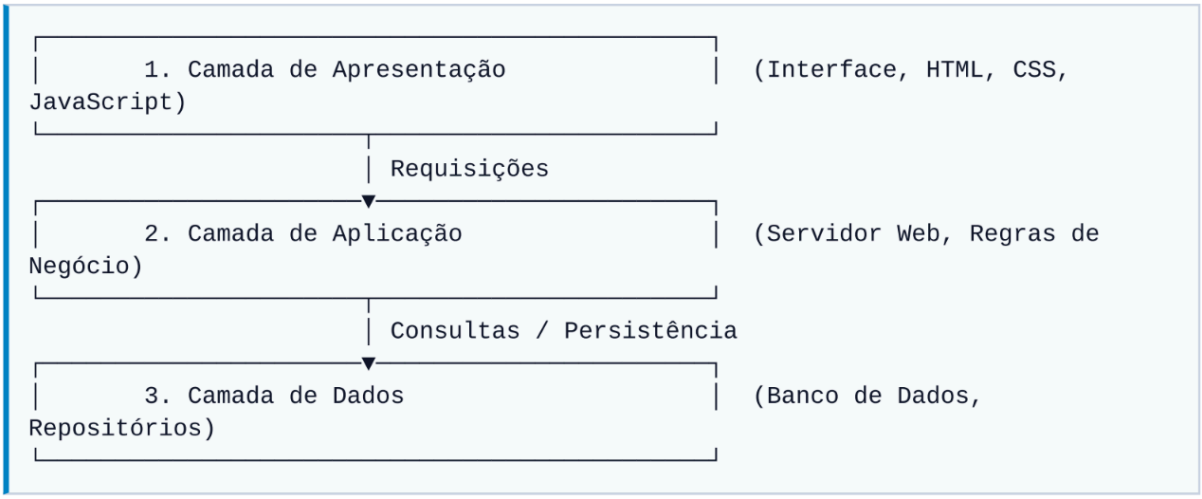
No cenário de controle de acessos, a arquitetura cliente-servidor atua como o mecanismo estrutural que dita o fluxo de dados e delimita as fronteiras de segurança, mitigando vulnerabilidades por meio da centralização das validações críticas no lado do servidor.

1. **Entrada de Dados:** O usuário insere suas credenciais de acesso (identificador único e credencial secreta) em um contêiner de formulário na interface gráfica. O navegador encapsula esses dados textuais e inicia a transmissão dos mesmos ao acionar o evento de envio.
2. **Encaminhamento Seguro:** Os dados encapsulados trafegam pela rede utilizando métodos de requisição cifrados (como o método POST via HTTPS), direcionando os parâmetros de identidade diretamente para os pontos de escuta do servidor web.
3. **Validação e Processamento:** O servidor de aplicação assume o controle do fluxo. Ele valida a estrutura e os campos obrigatórios da requisição e aciona a camada de persistência para recuperar o registro correspondente ao identificador informado. De posse do hash e dos parâmetros de segurança armazenados no banco, o servidor executa localmente a validação da credencial recebida por meio de um algoritmo criptográfico apropriado (como o Argon2id).
4. **Emissão da Resposta (Tratamento de Fluxo):**
    a. **Autenticação Positiva:** Caso as credenciais sejam válidas, o servidor retorna a identificação pública do usuário, permitindo que a aplicação cliente interprete o resultado e atualize sua interface conforme necessário.
    b. **Autenticação Negativa:** Diante de qualquer divergência ou ausência de registro, o servidor interrompe o fluxo e retorna um código de status de erro padronizado, impedindo o avanço do usuário sem expor detalhes estruturais da falha.

Dessa forma, o navegador permanece restrito à interação com o usuário, atuando como um ambiente de renderização visual e captura de eventos. As rotinas lógicas de validação, chaves criptográficas e políticas de segurança permanecem isoladas no ambiente controlado do servidor, resguardado o ecossistema contra engenharia reversa e manipulações locais na interface do usuário.

1.3. Organização Geral de uma Aplicação Web

Uma aplicação Web pode ser estruturada em camadas lógicas distintas. Essa separação de conceitos (Separation of Concerns) aprimora a manutenibilidade, a testabilidade e a escalabilidade do sistema (IEEE COMPUTER SOCIETY, 2024). Basicamente, essa tripartição divide os componentes em Camada de Apresentação, Camada de Aplicação e Camada de Dados, cada uma responsável por interagir com as informações de formas específicas no que rege o tipo de processamento a ser executado.



1.3.1. Camada de Apresentação

- **Responsabilidade:** Interface que se comunica diretamente com o usuário. Exibe dados e gerencia elementos interativos (formulários, botões e campos de entrada). Exemplo: O usuário acessa a página de login e informa as credenciais de e-mail e senha por meio de um formulário de acesso contendo elementos de interface, como os campos de entrada, botão de login e o Container que os abriga. Esta camada capta esses dados visuais e os encaminha de forma segura para o servidor de aplicação. **Tecnologias Típicas:** HTML (estrutura), CSS (estilização) e JavaScript (comportamento e interatividade).
- **Papel no Sistema:** Disponibiliza a interface de autenticação, captura os dados informados pelo usuário e faz o envio da requisição para o servidor realizar o processamento. Ademais, é por ela que o usuário recebe as informações que foram requisitadas por ele.

1.3.2. Camada de Aplicação

Responsabilidade: Concentra o processamento das requisições e a execução das regras de negócio e requisitos do sistema.

Exemplo: Após receber as credenciais, esta camada valida os campos obrigatórios, consulta o registro correspondente ao identificador informado e verifica a senha utilizando o algoritmo Argon2id, retornando o resultado da autenticação.

Componentes:

- **Servidor Web:** Trata o recebimento de conexões HTTP/HTTPS, roteamento e entrega de assets estáticos. Isso inclui o tráfego seguro dos dados que estarão vulneráveis durante o repasse de informações, sendo imprescindível a aplicação da inviolabilidade mediante os ataques cibernéticos.
- **Servidor de Aplicação:** Executa a lógica da aplicação, validações e orquestração dos serviços, atuando como a ponte inteligente que processa as regras de negócio do sistema.

Papel no Sistema: Recebe as credenciais enviadas pelo cliente, processa a lógica de autenticação e consulta a camada de persistência, garantindo a segurança e validando o acesso do usuário.

1.3.3. Camada de Dados

- **Responsabilidade:** Armazenamento persistente, indexação e disponibilização íntegra dos dados da aplicação, o que inclui uma alta taxa de respostas vindas das requisições e possíveis consultas na base de dados.
  Exemplo: Após receber a requisição de consulta da camada de aplicação contendo o identificador do usuário, esta camada realiza uma busca indexada em suas tabelas para localizar o registro correspondente, retornando ao servidor o hash da senha e os parâmetros criptográficos associados para que a validação seja processada localmente na aplicação.
- **Isolamento de Segurança:** O navegador jamais estabelece conexão direta com a camada de dados. Toda interação é intermediada pela camada de aplicação. Essa estrutura é fundamental para garantir a segurança das informações, pois reduz o risco de acessos diretos e vazamentos de dados.
- **Papel no Sistema:** Armazena de forma estruturada os registros cadastrais e as credenciais protegidas para validação durante o login, atuando como a camada final de persistência que garante a integridade e a confidencialidade das informações.

1.4. Protocolos e Comunicação no Ambiente Web

Para que os componentes das camadas de apresentação e de aplicação troquem informações pela rede, o ambiente operacional Web apoia-se em uma pilha de protocolos padronizados. Cada protocolo cumpre uma etapa técnica específica desde a localização do servidor até a formatação e entrega das mensagens de autenticação (MDN WEB DOCS, s.d.-a; MDN WEB DOCS, s.d.-b).

1. **DNS e Endereçamento IP:** Os dispositivos conectados à rede identificam-se por endereços numéricos denominados endereços IP (Internet Protocol). Quando a aplicação cliente precisa comunicar-se com o domínio do serviço, o navegador aciona o sistema DNS (Domain Name System), em conjunto com o sistema operacional, para converter o nome do domínio no endereço IP correspondente ao servidor Web de destino.
2. **TCP e TLS (Estabelecimento e Proteção da Conexão):** Localizado o endereço IP, o cliente estabelece um canal de transporte confiável com o servidor utilizando o protocolo TCP (Transmission Control Protocol), que oferece transporte confiável e ordenado dos dados, utilizando mecanismos de confirmação e retransmissão para lidar com perdas durante a comunicação. Sobre essa conexão, executa-se o aperto de mão (handshake) do protocolo TLS (Transport Layer Security), no qual o servidor apresenta seu certificado digital e ambos negociam chaves criptográficas para cifrar todo o tráfego subsequente.
3. **HTTP e HTTPS (Troca de Requisições e Respostas):** Estabelecido o túnel seguro, a comunicação passa a ocorrer na camada de aplicação por meio do protocolo HTTPS (HTTP sobre TLS). Sob esse modelo sem estado (stateless), o cliente submete uma requisição independente contendo os dados da operação e aguarda a resposta síncrona emitida pelo servidor (MDN WEB DOCS, s.d.-b).
4. **Estrutura das Mensagens HTTP:** As mensagens trocadas entre a aplicação cliente e o servidor seguem um formato estruturado composto por linha inicial, cabeçalhos (headers) e corpo (body). No escopo do Serviço de Autenticação Web, a invocação ocorre por meio do método POST direcionado ao caminho /auth/verify , trafegando objetos em notação JSON:



1.5. Considerações sobre o Ambiente Operacional

A estabilidade e a resiliência de uma aplicação Web estão diretamente vinculadas à interoperabilidade e cooperação contínua entre seus componentes, estendendo-se desde o navegador do cliente até os nós especializados de processamento e persistência de dados.

- **Arquitetura Base:** O modelo cliente-servidor estabelece as diretrizes fundamentais de acoplamento fraco, utilizando protocolos padronizados de rede para garantir que entidades distintas operem de forma síncrona e previsível.
- **Separação Lógica vs. Física:** A divisão estrutural em três camadas (Apresentação, Aplicação e Dados) constitui uma abstração lógica de responsabilidades. Na infraestrutura física, esses componentes podem coexistir em uma única instância computacional ou estarem distribuídos horizontalmente em arquiteturas heterogêneas, como clusters de computação em nuvem, orquestradores de contêineres e ecossistemas de microsserviços, variando de acordo com os requisitos de escalabilidade e disponibilidade do projeto.

No escopo específico deste sistema de autenticação, o mapeamento preciso dessas fronteiras operacionais é indispensável para mitigar as superfícies de ataque. Essa delimitação assegura a implementação de políticas rigorosas de segurança, criptografia de canal e isolamento de privilégios durante todo o ciclo de tráfego e tratamento de credenciais de acesso.

1.6. Componentes Operacionais do Ambiente Web

1.6.1. Navegador Web

No ambiente da internet, há diversos serviços que fazem a comunicação coexistir a ponto de compartilhar dados ininterruptamente. São necessárias infraestruturas inteiras, como servidores, satélites, cabos, roteadores, para que os dados trafeguem por essa rede de forma eficaz. Contudo, essa trafegabilidade geralmente ocorre mediante ao uso de softwares que intermediam a comunicação diretamente entre o cliente e servidor. Os mais usados são os navegadores web (como Google Chrome, Safari e Firefox).

Diferente dos motores de busca (como o Google ou Bing), que possuem funções de rastreabilidade e indexação sobre as páginas da internet, o papel do navegador é designado a requisitar, carregar, processar e renderizar as páginas web para a interação com o usuário. Sempre que um site é acessado, o navegador solicita os arquivos ao servidor e interpreta códigos em HTML (que estrutura o conteúdo), CSS (que define o design visual) e JavaScript (responsável pelo comportamento dinâmico e pela comunicação de dados de volta ao servidor). É através dessa interpretação visual que a maior parte dos usuários consome e interage com os dados na internet. A documentação da MDN Web Docs descreve o navegador como o software responsável por solicitar, interpretar e apresentar recursos da Web, em comunicação com servidores por meio de protocolos de rede (MDN WEB DOCS, s.d.-a).

- **Responsabilidades:** (1) Comunicação e rede: participa do processo de resolução de nomes (em conjunto com serviços do sistema operacional e servidores DNS) e estabelece conexões com os servidores para solicitar e receber os arquivos nos endereços corretos; (2) Leitura e desenho da página: interpreta os recursos recebidos do site, como textos, imagens e estilos, e transforma tudo isso na página organizada e visual que vemos na tela; (3) Memória e armazenamento: guarda pequenas informações no aparelho, como histórico, senhas e arquivos temporários, para que os sites carreguem mais rápido na próxima visita; e (4) Segurança e proteção: valida certificados TLS para autenticar a conexão segura com o servidor (embora isso não garanta, por si só, que o conteúdo do site seja legítimo), bloqueia arquivos suspeitos e mantém cada aba isolada para que um problema em uma página não comprometa o restante do sistema.
- **Funcionamento e processamento:** Ao digitar um endereço e confirmar, o navegador executa: (1) Busca do endereço na rede e abertura de canal seguro; (2) Pedido e entrega dos arquivos da página pelo servidor; (3) Organização do conteúdo separando estrutura e estilo visual; (4) Montagem na tela calculando tamanhos e posições; e (5) Ações e movimentos por meio de scripts responsáveis pela interatividade dinâmica.
- **Papel do Navegador na Autenticação:** Atua como intermediário em cinco passos: (1) Exibição do formulário com campos para credenciais; (2) Leitura das informações retendo temporariamente identificador e senha e ocultando visualmente os

caracteres da senha; (3) Envio protegido encaminhando as informações por conexão criptografada; (4) Recebimento da resposta com a identificação do usuário ou mensagem de erro; e (5) Exibição do resultado ao usuário.

1.6.2. Servidor Web

Enquanto o navegador atua na ponta do usuário solicitando recursos, o servidor web opera na outra extremidade como o sistema encarregado de armazenar, processar e disponibilizar os arquivos de um site. Trata-se de uma combinação entre infraestrutura física de alta capacidade e softwares especializados que permanecem em operação contínua para atender aos pedidos vindos da rede mundial de computadores. A MDN Web Docs diferencia o servidor Web como software e infraestrutura que recebem requisições HTTP e fornecem recursos aos clientes, diretamente ou por meio de aplicações (MDN WEB DOCS, s.d.-b).

- **Responsabilidades:** (1) Recepção e atendimento de solicitações nas portas de comunicação; (2) Gerenciamento de conexões simultâneas distribuindo recursos de processamento e memória; (3) Controle de acesso e segurança configurando e utilizando certificados digitais TLS (apresentando seu certificado ao cliente durante o estabelecimento da conexão HTTPS e realizando a terminação criptográfica) e aplicando regras de proteção; e (4) Registro e auditoria de atividades mantendo históricos detalhados de acesso.
- **Recebimento e encaminhamento de requisições:** Executa uma sequência de triagem: (1) Escuta e validação da conexão estabelecendo o canal seguro; (2) Análise do pedido recebido examinando cabeçalho, recurso e método HTTP; (3) Triagem entre conteúdo estático e dinâmico, atuando como Reverse Proxy e encaminhando pedidos que exigem processamento lógico à aplicação; e (4) Consolidação e envio da resposta: no Serviço de Autenticação Web proposto, o adaptador HTTP da aplicação (AuthHttpController) determina os códigos de status (200 , 400 , 401 , 500 e 503) conforme o resultado da autenticação, cabendo ao servidor Web (Reverse Proxy) encaminhar a resposta estruturada de volta ao navegador.

1.6.3. Servidor de Aplicação

Diferente do servidor web, que concentra sua atuação na entrega de arquivos prontos e no gerenciamento das conexões de rede, o servidor de aplicação constitui o núcleo lógico do sistema. É nesse ambiente que as regras de negócio são executadas, permitindo que as páginas respondam de forma dinâmica e personalizada às ações de cada usuário. Essa separação de responsabilidades pode ser compreendida a partir dos princípios de organização arquitetural apresentados no SWEBOK, especialmente quanto à definição de componentes e suas interações (IEEE COMPUTER SOCIETY, 2024).

- **Responsabilidades:** (1) Execução das regras de negócio aplicando validações e determinando o comportamento diante de cada solicitação; (2) Integração com bancos de dados; (3) Gerenciamento de estados: dependendo da arquitetura

adotada, servidores de aplicação podem manter informações de sessão e contexto dos usuários; entretanto, o Serviço de Autenticação Web proposto opera de forma estritamente stateless (sem estado de sessão entre requisições), conforme estabelecido na decisão arquitetural ADR03; e (4) Comunicação entre serviços.

- **Processamento da autenticação:** Conduz a verificação em quatro etapas: (1) Recepção e extração das credenciais validando presença e formato dos campos obrigatórios; (2) Consulta e verificação criptográfica consultando o registro do usuário e utilizando o algoritmo Argon2id para verificar a correspondência entre a senha informada e o hash armazenado, sem persistir ou registrar a senha original (executando verificação contra um hash fictício pré-configurado caso o identificador não seja localizado, conforme a decisão ADR01); (3) Decisão de autenticação: o escopo do serviço restringe-se exclusivamente à verificação das credenciais (CON01), sem avaliar status cadastral da conta, bloqueios por tentativas ou permissões de acesso, considerando unicamente a existência do registro e a validade criptográfica da senha; e (4) Retorno do resultado da autenticação: retorna a identificação pública do usuário (id e identifier) em caso válido, ou decisão de rejeição convertida em resposta HTTP genérica (HTTP 401), sem revelar se o identificador existe na base de dados e sem emitir identificadores de sessão ou tokens de acesso persistentes (CON01).

1.6.4. Banco de Dados

O banco de dados constitui a camada permanente de memória de um sistema, sendo o repositório estruturado onde as informações são registradas e organizadas. Enquanto os servidores web e de aplicação atuam no tráfego e no processamento temporário das requisições, esse ambiente dispõe de mecanismos voltados à preservação da integridade, consistência e disponibilidade dos dados essenciais ao funcionamento da aplicação. As práticas de armazenamento de senhas devem considerar mecanismos de hash específicos para essa finalidade, conforme as recomendações de segurança da OWASP (OWASP FOUNDATION, s.d.).

- **Armazenamento de usuários:** Em bancos de dados relacionais, os registros são estruturados em tabelas e campos com restrições de unicidade. As senhas não são armazenadas em texto puro, sendo processadas por algoritmos de hash específicos como o Argon2id com salt individual. Embora bancos corporativos possam conter atributos adicionais de status ou permissões, o Serviço de Autenticação Web proposto limita-se estritamente à leitura dos atributos necessários à verificação de credenciais (CON01).
- **Consultas e recuperação de credenciais:** No Serviço de Autenticação Web proposto, a base de usuários é um repositório externo preexistente e acessado exclusivamente para leitura, sem operações de cadastro ou modificação por parte do serviço, conforme estabelecido na premissa arquitetural PRE01. O banco recebe a instrução de leitura, realiza localização indexada do registro pelo identificador, extrai seletivamente os atributos necessários (id , identifier e password_hash) e devolve-os com segurança ao componente de persistência da aplicação.

1.6.5. Integração entre os Componentes

O funcionamento completo da autenticação depende da comunicação coordenada entre todas as camadas da arquitetura, formando um fluxo contínuo de solicitação, processamento e resposta, seguindo o modelo de requisição e resposta empregado pelo HTTP (MDN WEB DOCS, s.d.-b): (1) envio da requisição HTTPS pelo navegador; (2) recepção, terminação TLS e encaminhamento pelo servidor Web (Reverse Proxy) ao servidor de aplicação; (3) validação dos campos e consulta somente leitura ao banco de dados preexistente; (4) verificação criptográfica via Argon2id e emissão do parecer de autenticação; e (5) retorno e exibição do resultado ao usuário.

SEÇÃO 2 — PROJETO DA ARQUITETURA E DESENHO DETALHADO

2.1. Especificação de Requisitos de Software e Direcionadores Arquiteturais

- **Autor:** David Daves Ferreira Pinto
- **Revisão assistida por IA:** ChatGPT (OpenAI)
- **Referências deste Item:** [ISO/IEC/IEEE 12207, 2017]; [SWEBOK V4.0, 2024]; [FIELDING; TAYLOR, 2002]; [BASS et al., 2021].

2.1.1. Escopo e Fronteiras do Sistema

Objetivo

Este documento especifica os requisitos funcionais, não funcionais e as decisões arquiteturais do serviço de autenticação de usuários para aplicações Web.

Escopo

O escopo do sistema limita-se à verificação de credenciais de usuários. O fluxo compreende receber as credenciais do cliente, validar a estrutura da requisição, consultar o usuário no banco de dados, comparar a senha informada com o hash armazenado e retornar o resultado da autenticação (sucesso com os dados do usuário ou erro padronizado).

Fora de Escopo

Estão expressamente fora do escopo do sistema as seguintes funcionalidades:

- Cadastro, alteração ou exclusão de usuários;
- Gestão de status de conta (ativação, bloqueio ou suspensão);
- Recuperação ou redefinição de senhas;
- Emissão de tokens de sessão (JWT/OAuth), refresh tokens ou controle de acesso (RBAC).

2.1.2. Partes Interessadas e Interesses Arquiteturais

Alinhado com a norma ISO/IEC/IEEE 42010 e o SWEBOK v4.0, o projeto atende aos seguintes interesses das partes interessadas:

Necessidades das Partes Interessadas

- **STK01 — Segurança de Credenciais:** Garantir a proteção estrita de senhas e credenciais em trânsito e em repouso, prevenindo vazamentos, registro indevido em logs e ataques de enumeração ou temporais.
- **STK02 — Contrato de API Padronizado:** Disponibilizar uma interface HTTP/REST clara, previsível e baseada em JSON para facilitar a integração simples com aplicações clientes Web.
- **STK03 — Testabilidade Isolada:** Fornecer uma arquitetura modular que permita testar as regras de negócio de autenticação de forma automatizada e totalmente desprovida de dependências de infraestrutura (banco de dados, rede, HTTP).
- **STK04 — Desempenho sob Carga:** Assegurar respostas rápidas e alta disponibilidade do serviço de autenticação mesmo sob vazão constante de requisições em ambiente de produção.
- **Aplicações Clientes:** Necessitam de contrato de API claro, respostas previsíveis e mensagens de erro padronizadas.
- **Engenheiros de Software e Arquitetos:** Requerem código modular, desacoplamento entre camadas e facilidade para testes automatizados.
- **Segurança e Operações:** Exigem proteção de credenciais em trânsito e em repouso, proibição de logs com senhas em texto plano e resiliência a falhas de infraestrutura.
- **STK05 — Verificação de Identidade:** Garantir que o sistema confirme com precisão a autenticidade das credenciais informadas pelo usuário antes de conceder acesso às respostas de sucesso.
- **STK06 — Identificação do Usuário:** Retornar dados essenciais e seguros de identificação do usuário quando a autenticação for bem-sucedida, sem expor informações sensíveis.
- **STK07 — Comunicação de Falhas:** Fornecer mensagens de erro estruturadas e previsíveis em cenários de credenciais inválidas, falhas de validação ou indisponibilidade de infraestrutura.

2.1.3. Requisitos Funcionais

Especificação dos Requisitos Funcionais (RFs)

Os requisitos funcionais definem os comportamentos atômicos do sistema:

- **RF01 — Receber Requisição de Autenticação:** Ao receber uma solicitação no endpoint POST /auth/verify, o sistema deve receber os campos obrigatórios de identificador (string não vazia resolvida pelo repositório) e senha fornecidos pelo cliente web.

  [Rationale: Capturar os dados de entrada de credenciais para o fluxo de autenticação | Origem: Enunciado da APS | Referência metodológica: ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Alta | Estabilidade: Alta | Verificação: Test]
- **RF02 — Validar Sintaxe dos Dados de Entrada:** Quando a requisição for recebida, o sistema deve validar a presença, os tipos de dados e a não-vacuidade dos campos obrigatórios identificador e senha.

  [Rationale: Bloquear requisições malformadas na camada de controle antes do processamento | Origem: Enunciado da APS | Referência metodológica:

ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Alta | Estabilidade: Alta | Verificação: Test]

- **RF03 — Consultar Cadastro do Usuário:** Após a validação sintática bem-sucedida, o sistema deve pesquisar a conta do usuário no repositório de dados utilizando o identificador informado.
  [Rationale: Localizar o registro persistido e recuperar os parâmetros do hash para validação | Origem: Enunciado da APS | Referência metodológica: ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Alta | Estabilidade: Alta | Verificação: Test]
- **RF04 — Executar Verificação de Senha:** Quando a conta do usuário for localizada, o sistema deve comparar a senha informada com o hash armazenado utilizando o mecanismo de verificação de senhas compatível com o algoritmo empregado no armazenamento (Argon2id).
  [Rationale: Confirmar a autenticidade da credencial fornecida pelo cliente mediante algoritmo adaptativo | Origem: Enunciado da APS | Referência metodológica: ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Alta | Estabilidade: Alta | Verificação: Test]
- **RF05 — Emitir Resposta de Autenticação Bem-Sucedida:** Quando a verificação de senha for confirmada com sucesso, o sistema deve retornar a resposta HTTP 200 OK contendo os dados de identificação do usuário (como id e e-mail), sem expor credenciais ou hashes.
  [Rationale: Notificar o cliente quanto ao sucesso do procedimento e disponibilizar os dados básicos | Origem: Enunciado da APS | Referência metodológica: ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Alta | Estabilidade: Alta | Verificação: Test]
- **RF06 — Emitir Resposta de Credenciais Inválidas:** Quando o usuário não for encontrado ou a senha for divergente, o sistema deve retornar a resposta HTTP 401 Unauthorized com mensagem padronizada.
  [Rationale: Informar a falha de autenticação sem revelar a existência do identificador e mitigar diferenças temporais observáveis | Origem: Enunciado da APS | Referência metodológica: ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Alta | Estabilidade: Alta | Verificação: Test]
- **RF07 — Emitir Resposta de Falha de Validação:** Quando os dados de entrada descumprirem as regras sintáticas, o sistema deve retornar a resposta HTTP 400 Bad Request detalhando os erros detectados.
  [Rationale: Notificar o cliente sobre falhas no contrato de requisição | Origem: Enunciado da APS | Referência metodológica: ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Alta | Estabilidade: Alta | Verificação: Test]
- **RF08 — Tratamento de Falhas de Infraestrutura:** Em caso de indisponibilidade ou estouro de tempo limite na comunicação com a base de dados, o sistema deve retornar a resposta HTTP 503 Service Unavailable.
  [Rationale: Isolar exceções técnicas e garantir resposta estruturada perante falhas externas | Origem: Enunciado da APS | Referência metodológica: ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Média | Estabilidade: Alta | Verificação: Test]

Critérios de Aceitação e Fluxos Processuais

A Tabela 1 especifica os critérios de aceitação para validação automatizada e testes de aceitação dos requisitos funcionais.

Tabela 1 — Critérios de Aceitação e Comportamento Esperado por Cenário

ID	Cenário / Fluxo	Entrada	Condição de Execução	Resultado Esperado (Saída)	Ref.
CA01	**Autenticação com Sucesso**	*identifier* e *password* válidos	Usuário cadastrado e hash de senha correspondente	Status *HTTP 200 OK* e dados do usuário	RF01, RF02, RF03, RF04, RF05
CA02	**Usuário Não Encontrado**	*identifier* e *password* preenchidos	Identificador não cadastrado no banco de dados	Status *HTTP 401 Unauthorized* e mensagem de erro de credenciais	RF01, RF02, RF03, RF06
CA03	**Senha Incorreta**	*identifier* e *password* preenchidos	Usuário localizado, porém a senha não corresponde ao hash	Status *HTTP 401 Unauthorized* e mensagem de erro de credenciais (mesma resposta do CA02)	RF01, RF02, RF03, RF04, RF06
CA04	**Dados de Entrada Inválidos**	Payload sem *identifier*, sem *password* ou JSON malformado	Falha na validação dos campos obrigatórios	Status *HTTP 400 Bad Request* e detalhes dos erros de validação	RF01, RF02, RF07
CA05	**Falha do Banco de Dados**	*identifier* e *password* preenchidos	Indisponibilidade de conexão ou timeout na consulta ao repositório	Status *HTTP 503 Service Unavailable* e mensagem genérica de erro de serviço no cenário de indisponibilidade do banco de dados	RF01, RF03, RF08

Fonte: Elaborada pelo autor com base nos requisitos funcionais e critérios de aceitação definidos nesta seção.

2.1.4. Requisitos Não Funcionais (RNFs) e Decisões Arquiteturais (ADRs)

Esta seção separa os Requisitos Não Funcionais (metas de qualidade verificáveis) das Decisões Arquiteturais (mecanismos de projeto adotados).

Segurança (RNF01 / ADR01)

- **Requisito Não Funcional (RNF01 / ASR01):** O sistema deve garantir a confidencialidade das credenciais em trânsito, proibir o registro de senhas em texto claro em logs e exigir a mitigação de diferenças temporais observáveis para prevenir ataques temporais e de enumeração de usuários.
- **Decisão Arquitetural (ADR01):** Obrigatoriedade do protocolo HTTPS (TLS 1.3) na camada de transporte. O algoritmo de hash adotado é o Argon2id. Para mitigar diferenças temporais observáveis, o sistema executará um cálculo de hash fictício equivalente quando o identificador não for localizado na base de dados.

Contrato de API e Comunicação (RNF02 / ADR02)

- **Requisito Não Funcional (RNF02):** O sistema deve disponibilizar uma interface HTTP padronizada baseada em JSON para integração simples com clientes Web.
- **Decisão Arquitetural (ADR02):** Definição do endpoint *POST /auth/verify* aceitando o cabeçalho *Content-Type: application/json*. Especificação detalhada das estruturas de payload:
    - **Método e Endpoint:** *POST /auth/verify* (Cabeçalho obrigatório: *Content-Type: application/json*).
    - **Formato da Requisição (Entrada):**

```
JSON
{
  "identifier": "usuario@dominio.com",
  "password": "SenhaSubmetidaPeloCliente"
}
```

    - **Resposta de Sucesso — Deferimento (HTTP 200 OK):**

```
JSON
{
  "status": "AUTHENTICATED",
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "identifier": "usuario@dominio.com"
  }
}
```

    - **Resposta de Erro de Autenticação (HTTP 401 Unauthorized):**

```
JSON
{
  "status": "UNAUTHENTICATED",
  "error": "INVALID_CREDENTIALS",
  "message": "Credenciais de autenticacao invalidas."
```

```
}
```

- **Resposta de Erro de Validação (HTTP 400 Bad Request):**

```
JSON
{
  "status": "BAD_REQUEST",
  "error": "VALIDATION_ERROR",
  "message": "Os campos 'identifier' e 'password' sao obrigatorios."
}
```

- **Resposta de Erro Técnico (HTTP 503 Service Unavailable):**

```
JSON
{
  "status": "ERROR",
  "error": "SERVICE_UNAVAILABLE",
  "message": "Servico temporariamente indisponivel para processamento."
}
```

Desempenho, Disponibilidade e Concorrência (RNF03 / ADR03)

- **Requisito Não Funcional (RNF03 / ASR03 — Desempenho e Capacidade):** Sob uma carga constante de 100 requisições por segundo mantida durante um período contínuo de 10 minutos (conforme especificado no Plano de Verificação), o sistema deve manter o tempo de resposta no percentil 95 (P95) inferior a 200 milissegundos e a taxa de erros técnicos (HTTP 5xx) inferior a 0,1% em um ambiente de referência composto por 2 vCPUs e 4 GB de RAM.
  [Rationale: Garantir capacidade de resposta e vazão dentro das métricas de desempenho sob carga operacional especificada | Origem: Enunciado da APS | Referência metodológica: ISO/IEC/IEEE 29148:2018, seção 9.6 | Prioridade: Alta | Estabilidade: Alta | Verificação: Test]
- **Decisão Arquitetural (ADR03):** Arquitetura stateless no servidor de aplicação, pool de conexões TCP gerenciado com a base de dados configurado com limite de tempo de conexão (connect-timeout) de 2000ms e tempo de leitura (read-timeout) de 3000ms como mecanismos de controle de falhas e contenção de esperas prolongadas, além de tratamento global de exceções de infraestrutura, enquanto o cumprimento da meta de P95 inferior a 200ms será verificado por testes de carga.

Modificabilidade e Testabilidade (RNF04 / ADR04)

- **Requisito Não Funcional (RNF04 / ASR04):** A arquitetura do sistema deve focar na isolação de testes unitários da regra de negócio de autenticação, permitindo a execução de testes automatizados sem dependência direta de banco de dados, rede ou servidor HTTP.
- **Decisão Arquitetural (ADR04):** Aplicação da Arquitetura Hexagonal através de interfaces abstratas para o repositório de usuários (UserRepositoryPort) e verificador de senha (PasswordVerifierPort).

Requisitos Lógicos de Dados e Limites de Interface Externa (RLD01 / LIE01 / LIE02)
Conforme as diretrizes das seções 9.6.11 e 9.6.15 da ISO/IEC/IEEE 29148, definem-se as especificações lógicas dos dados e as fronteiras das interfaces externas do serviço:

- **RLD01 — Schema do Usuário (Repositório de Dados):** A entidade lógica do usuário deve conter os campos: id (UUID v4, 36 caracteres, chave primária), identifier (string não vazia resolvida pelo repositório, única) e password_hash (String encodada em ASCII/Base64 contendo os parâmetros do algoritmo *Argon2id*, salt e hash gerado).
- **LIE01 — Interface HTTP/JSON Externa:** O serviço comunica-se via protocolo HTTPS (TLS 1.3) enviando e recebendo payloads serializados em formato JSON (UTF-8). O payload de entrada aceita no máximo 4 KB por requisição para prevenção de consumo excessivo de memória.
- **LIE02 — Interface de Repositório de Persistência:** Define o contrato abstrato de comunicação lógica entre a camada de aplicação e o repositório de persistência de dados para consulta e recuperação dos registros de usuários.

2.1.5. Restrições e Premissas

Restrições de Escopo

- **CON01 — Limitação de Escopo:** Proibida a inclusão de endpoints ou esquemas de dados que não pertençam estritamente ao fluxo de verificação de credenciais.

Restrições do Ambiente Web

- **CON02 — Ambiente Web:** O serviço deve funcionar integrado a servidores web proxy/reverse proxy com suporte a terminação TLS, conforme especificação detalhada no documento de ambiente Web da aplicação.

Premissas Arquiteturais

- **PRE01 — Base de Dados Existente:** Assume-se que a base de dados já possui usuários cadastrados com senhas salvas em hash de formato compatível com o algoritmo configurado.

2.1.6. Matriz de Rastreabilidade

A Tabela 2 apresenta o mapeamento completo entre as etapas do processo, requisitos funcionais atômicos, critérios de aceitação e decisões de arquitetura.

Tabela 2 — Matriz de Rastreabilidade Geral do Documento

STK	Req.	Descrição Atômica	Critérios (CA)	ADR / Interface	Verif.	Prior.	Etapa de Domínio
STK02	**RF01**	Receber requisição de autenticação (identifier e password)	CA01, CA02, CA03, CA04, CA05	ADR02, LIE01	Test	Alta	Entrada de dados
STK02	**RF02**	Validar sintaxe dos dados de entrada	CA01, CA02, CA03, CA04	ADR02, LIE01	Test	Alta	Entrada de dados
STK03	**RF03**	Consultar cadastro do usuário	CA01, CA02, CA03, CA05	ADR04, RLD01, LIE02	Test	Alta	Consulta usuário

STK	Req.	Descrição Atômica	Critérios (CA)	ADR / Interface	Verif.	Prior.	Etapa de Domínio
STK01, STK03	**RF04**	Executar verificação de senha	CA01, CA03	ADR01, ADR04	Test	Alta	Verificação senha
STK02	**RF05**	Emitir resposta de autenticação bem-sucedida (200 OK)	CA01	ADR02	Test	Alta	Retorno resposta
STK01, STK02	**RF06**	Emitir resposta de credenciais inválidas (401 Unauthorized)	CA02, CA03	ADR01, ADR02	Test	Alta	Retorno resposta
STK02	**RF07**	Emitir resposta de falha de validação (400 Bad Request)	CA04	ADR02	Test	Alta	Retorno resposta
STK04	**RF08**	Tratamento de falhas de infraestrutura (503 Service Unavailable)	CA05	ADR03, LIE02	Test	Média	Retorno resposta

Fonte: Elaborada pelo autor com base em [SWEBOK V4.0, 2024] e [KRUCHTEN, 1995].

2.2. Descrição Arquitetural e Especificação dos Componentes

- **Autor:** David Daves Ferreira Pinto
- **Revisão assistida por IA:** ChatGPT (OpenAI)
- **Referências deste Item:** (COCKBURN, 2005); (IEEE COMPUTER SOCIETY, 2024); (MDN WEB DOCS, s.d.-a); (MDN WEB DOCS, s.d.-b).

2.2.1. Visão Geral da Arquitetura

Objetivo da Arquitetura

A presente descrição arquitetural estabelece a organização estrutural, os contratos de comunicação e os limites operacionais do **Serviço de Autenticação Web**, traduzindo os requisitos funcionais (RF01 a RF08) e não funcionais (RNF01 a RNF04) definidos na Especificação de Requisitos (Subseção 2.1) em componentes de software claramente delimitados. Em conformidade com os conceitos e práticas da área de conhecimento de Arquitetura de Software apresentados pela IEEE Computer Society (2024), este documento estabelece a relação entre a especificação de requisitos e as decisões que orientam a implementação, direcionando o sistema ao cumprimento de sua finalidade: receber credenciais de acesso, consultar a existência do usuário, verificar a senha em relação ao hash armazenado e retornar a decisão correspondente.

Para além do comportamento funcional, a arquitetura foi desenhada para estruturar a verificação de credenciais diante das exigências e padrões de comunicação do ambiente operacional Web (CON02) (MDN WEB DOCS, s.d.-a; MDN WEB DOCS, s.d.-b). O projeto estabelece mecanismos arquiteturais destinados a atender aos requisitos de qualidade e às restrições operacionais do sistema — abrangendo a proteção das credenciais (RNF01/ADR01), a padronização da comunicação HTTP/JSON (RNF02/ADR02), o desempenho e a capacidade operacional (RNF03/ADR03) e a testabilidade e modificabilidade (RNF04/ADR04) —, permitindo sua posterior verificação por meio de testes

e avaliações técnicas, amparadas pelas visões arquiteturais descritas no modelo C4 (BROWN, 2018; IEEE COMPUTER SOCIETY, 2024).

Estilo Arquitetural Adotado

A organização interna da aplicação adota o estilo de **Arquitetura Hexagonal**, igualmente denominado **Portas e Adaptadores (*Ports and Adapters*)** (COCKBURN, 2005), conforme registrado na decisão arquitetural ADR04 e alinhado aos princípios de modularidade e separação de preocupações (*separation of concerns*) descritos pela IEEE Computer Society (2024). Sob esse paradigma, o software é particionado em um núcleo central de aplicação — detentor exclusivo da lógica do caso de uso de autenticação — e uma camada periférica composta por adaptadores técnicos que medeiam a interação do sistema com o ambiente externo.

A concretização desse isolamento ocorre por meio da inversão de dependências: o núcleo da aplicação não conhece nem importa implementações concretas de servidores HTTP, bancos de dados ou bibliotecas criptográficas, comunicando-se exclusivamente através de contratos abstratos (interfaces) denominados portas (COCKBURN, 2005). As portas de entrada (*Driving Ports*) definem como agentes externos acionam o núcleo, ao passo que as portas de saída (*Driven Ports*) especificam as capacidades que o núcleo exige da infraestrutura para concluir seu processamento. A Tabela 3 sintetiza a distribuição dos elementos do projeto segundo essa estrutura, em alinhamento com o modelo C4 proposto por Brown (2018) e representado no Diagrama C4 de Componentes (Figura 3).

Tabela 3 — Organização dos Elementos na Arquitetura Hexagonal

Camada ou Elemento	Componente no Projeto e Responsabilidade Síntese
Adaptador Primário	`AuthHttpController` — recebe e valida sintaticamente as requisições HTTP/JSON, traduzindo o desfecho do caso de uso nos códigos de status previstos (MDN WEB DOCS, s.d.-b).
Porta de Entrada	`AuthenticationUseCasePort` — contrato abstrato que expõe a operação do caso de uso de verificação de credenciais.
Núcleo da Aplicação	`AuthenticationService` — coordena o fluxo de autenticação, determinando a validade das credenciais (IEEE COMPUTER SOCIETY, 2024).
Portas de Saída	`UserRepositoryPort` e `PasswordVerifierPort` — contratos abstratos requeridos pelo núcleo para leitura de dados e verificação criptográfica.
Adaptadores Secundários	`DatabaseUserRepositoryAdapter` e `Argon2PasswordVerifierAdapter` — implementam as portas de saída, integrando o SGBD externo e o algoritmo Argon2id.

Fonte: Elaborada pelo autor com base no Diagrama C4 de Componentes (Figura 3).

Justificativa da Arquitetura Hexagonal

A escolha da Arquitetura Hexagonal justifica-se pela necessidade de proteger a regra decisória de autenticação contra o acoplamento direto a tecnologias de infraestrutura e protocolos de rede (COCKBURN, 2005; IEEE COMPUTER SOCIETY, 2024). Em serviços de segurança, a regra que determina quando um usuário deve ser considerado autenticado ou rejeitado precisa permanecer clara, estável e independente de detalhes de transporte ou armazenamento. Ao isolar o `AuthenticationService` atrás de interfaces abstratas, evita-se que particularidades do protocolo Web (como cabeçalhos HTTP, serialização JSON ou códigos de status `200`, `400`, `401`, `500` e `503`) se misturem à lógica central de verificação, mantendo as responsabilidades de comunicação restritas ao `AuthHttpController`.

Adicionalmente, esse desacoplamento atende diretamente às diretrizes de testabilidade e modificabilidade (RNF04/ADR04) (IEEE COMPUTER SOCIETY, 2024). Essa organização permite verificar isoladamente as regras do caso de uso de autenticação por meio de testes unitários automatizados, utilizando *mocks* ou *stubs* para simular as dependências externas (`UserRepositoryPort` e `PasswordVerifierPort`) (COCKBURN, 2005). A validação integral dos critérios de aceitação (CA01 a CA05), incluindo os contratos HTTP no adaptador primário e as integrações reais de infraestrutura nos adaptadores secundários, deverá ser complementada por testes de integração e de aceitação. Do mesmo modo, eventuais substituições do banco de dados ou da biblioteca criptográfica podem ser realizadas nos adaptadores secundários, sem exigir alterações no núcleo, desde que os contratos das portas permaneçam compatíveis.

Fronteiras e Dependências Externas

Conforme formalizado no Diagrama C4 de Contexto (Figura 1) e no Diagrama C4 de Contêineres (Figura 2), a fronteira arquitetural do **Serviço de Autenticação Web** abrange o contêiner de borda (**Servidor Web / Reverse Proxy**) e o contêiner de execução da regra de negócio (**Aplicação de Autenticação**), alinhando-se aos princípios de estruturação cliente-servidor e intermediação na Web descritos em MDN Web Docs (s.d.-a). O escopo funcional contido nessa fronteira submete-se rigorosamente à restrição CON01 e às diretrizes de delimitação funcional (IEEE COMPUTER SOCIETY, 2024), limitando-se à verificação de credenciais e vedando qualquer operação de cadastro, edição cadastral, controle de status de conta (ativo/inativo) ou gerenciamento de sessões pós-autenticação.

Fora dos limites do sistema situam-se dois elementos externos que interagem com a solução:

- **1. Aplicação Cliente (Consumidor Externo):** Sistema executado no ambiente do usuário (como uma aplicação em navegador Web) que coleta o identificador e a senha informados pelo ator humano e solicita a autenticação (`POST /auth/verify`) via HTTPS/JSON ao serviço (MDN WEB DOCS, s.d.-b),

recebendo de volta a identificação do usuário ou a mensagem de erro correspondente.

- **2. Base de Usuários (Dependência Externa Preexistente):** Em observância à premissa arquitetural PRE01, o repositório que armazena os registros de usuários e seus respectivos hashes de senha pertence a um sistema externo preexistente, sem responsabilidade de provisionamento ou gerência por parte do serviço em análise. A **Aplicação de Autenticação** interage com essa base exclusivamente em modo de leitura (*read-only*) por intermédio do `DatabaseUserRepositoryAdapter`, tratando eventuais falhas de conectividade ou *timeout* como erros de infraestrutura (RF08) distintos da rejeição por credenciais inválidas (RF06).

2.2.2. Organização Arquitetural

Organização em Contêineres

A visão de contêineres, representada no Diagrama C4 de Contêineres (Figura 2), apresenta a distribuição das responsabilidades operacionais entre as unidades de execução do sistema para atender à restrição do ambiente Web (CON02) e ao requisito de segurança em trânsito (RNF01/ADR01) (MDN WEB DOCS, s.d.-a; MDN WEB DOCS, s.d.-b).

A comunicação HTTP entre o **Servidor Web / Reverse Proxy** e a **Aplicação de Autenticação** ocorre exclusivamente em uma rede interna isolada e controlada, sem exposição direta à Internet. Caso atravesse redes não confiáveis, essa comunicação deverá ser protegida por TLS, preservando o requisito RNF01. A Tabela 4 identifica cada contêiner e sistema interagente, detalhando seu papel operacional.

Tabela 4 — Especificação dos Contêineres e Sistemas Interagentes (C4 Nível 2)

Elemento / Contêiner	Escopo	Protocolo de Comunicação	Responsabilidade Operacional
Aplicação Cliente	Externo	`HTTPS / JSON`	Coleta as credenciais informadas pelo usuário, submete a requisição de autenticação ao serviço e conduz o fluxo subsequente conforme a decisão retornada.
Servidor Web / Reverse Proxy	Interno	`HTTPS` (Externa) e `HTTP` (Rede Interna Isolada)	Ponto de entrada de rede do sistema. Recebe conexões externas seguras, executa a terminação criptográfica TLS (ADR01) e encaminha as requisições HTTP para a aplicação em rede interna controlada (MDN WEB DOCS, s.d.-b).
Aplicação de Autenticação	Interno	`HTTP / JSON` e `TCP (SGBD)`	Contêiner de aplicação *stateless* (ADR03) que expõe a interface HTTP

Elemento / Contêiner	Escopo	Protocolo de Comunicação	Responsabilidade Operacional
			interna, valida os dados recebidos, coordena a consulta ao usuário e a verificação da senha, e emite o resultado da autenticação.
Base de Usuários	Externo	`TCP / Driver SGBD` (Somente Leitura)	Repositório preexistente (PRE01) que armazena de forma persistente os registros de identificação e os hashes de senhas dos usuários.

Fonte: Elaborada pelo autor com base no Diagrama C4 de Contêineres (Figura 2).

Componentes e Responsabilidades

No interior do contêiner **Aplicação de Autenticação**, os módulos de código organizam-se conforme o Diagrama C4 de Componentes (Figura 3). As portas (`AuthenticationUseCasePort`, `UserRepositoryPort` e `PasswordVerifierPort`) correspondem a contratos abstratos de comunicação entre componentes dentro do mesmo espaço de endereçamento, não constituindo unidades independentes de execução (COCKBURN, 2005). A Tabela 5 define as responsabilidades e as dependências estáticas de cada componente.

Tabela 5 — Especificação dos Componentes Internos da Aplicação (C4 Nível 3)

Componente	Classificação Arquitetural	Dependências Declaradas	Responsabilidade Técnica Detalhada
`AuthHttpController`	Adaptador Primário (Entrada HTTP)	`AuthenticationUseCasePort`	Recebe a requisição HTTP/JSON encaminhada pelo Reverse Proxy, valida a presença e o formato dos campos obrigatórios, aciona a porta de entrada e converte o resultado ou exceções nas respostas HTTP previstas (`200`, `400`, `401`, `503` ou `500`).
`AuthenticationUseCasePort`	Porta de Entrada (Interface)	Nenhuma (Contrato do Núcleo)	Define a assinatura abstrata de invocação do caso de uso de verificação de credenciais, isolando o controlador HTTP da implementação do serviço.
`AuthenticationService`	Serviço de Aplicação / Caso de Uso	`UserRepositoryPort`, `PasswordVerifierPort`	Implementa `AuthenticationUseCasePort`. Consulta o usuário, solicita a verificação da senha (aplicando verificação com hash fictício

Componente	Classificação Arquitetural	Dependências Declaradas	Responsabilidade Técnica Detalhada
			pré-configurado quando o identificador não existe) e determina a aprovação ou rejeição da autenticação.
`UserRepositoryPort`	Porta de Saída (Interface)	Nenhuma (Contrato do Núcleo)	Define a operação abstrata de busca de usuário por identificador, desacoplando o núcleo da tecnologia de banco de dados.
`PasswordVerifierPort`	Porta de Saída (Interface)	Nenhuma (Contrato do Núcleo)	Define a operação abstrata de verificação criptográfica de equivalência entre a senha informada e o hash armazenado.
`DatabaseUserRepositoryAdapter`	Adaptador Secundário (Persistência)	`UserRepositoryPort` e Driver SGBD	Implementa `UserRepositoryPort`. Executa a consulta somente leitura na Base de Usuários externa e converte falhas de conexão ou *timeout* em erros de infraestrutura reconhecíveis pela aplicação.
`Argon2PasswordVerifierAdapter`	Adaptador Secundário (Criptografia)	`PasswordVerifierPort` e Biblioteca Argon2id	Implementa `PasswordVerifierPort`. Executa a verificação segura de senhas utilizando o algoritmo Argon2id a partir dos parâmetros de custo e *salt* registrados no próprio hash.

Fonte: Elaborada pelo autor com base no Diagrama C4 de Componentes (Figura 3).

2.2.3. Interfaces e Comportamento

Contratos das Portas

Para assegurar a consistência na integração entre os componentes e orientar a implementação e os testes automatizados (IEEE COMPUTER SOCIETY, 2024), esta subseção formaliza os contratos das interfaces internas e das estruturas de dados do núcleo da aplicação. Adota-se a convenção de tipos `String`, `Boolean`, `Throwable` e `Optional<T>` (indicando valor opcional que pode estar presente ou ausente/vazio).

O contrato externo HTTP/JSON (`POST /auth/verify`) permanece regido pelas definições da Especificação de Requisitos (Subseção 2.1), cabendo ao `AuthHttpController` realizar o mapeamento explícito entre os nomes internos do núcleo (como `userId` em `AuthDecisionResult`) e os campos externos do payload JSON (como `user.id` na resposta `HTTP 200 OK`), conforme detalhado nas Tabelas 6 e 7.

Tabela 6 — Estruturas de Dados e Exceções de Contrato do Núcleo

Estrutura ou Tipo	Atributos e Tipagem	Semântica e Regras de Integridade
`AuthCredentialsInput`	`identifier: String`; `password: String`	Dados de entrada fornecidos à porta do caso de uso. Pré-condição: ambos os atributos devem ser não nulos, não vazios e validados sintaticamente pelo adaptador primário.
`UserRecord`	`id: String`; `identifier: String`; `passwordHash: String`	Entidade de leitura retornada pelo repositório quando o identificador é localizado. `passwordHash` contém a representação completa do Argon2id (algoritmo, versão, parâmetros de custo, *salt* e resumo).
`AuthDecisionResult`	`authenticated: Boolean`; `userId: Optional<String>`; `identifier: Optional<String>`	Resultado do caso de uso. Quando `authenticated == true`, `userId` (recebendo `userRecord.id`) e `identifier` estão obrigatoriamente presentes (RF05). Quando `authenticated == false`, ambos permanecem vazios (`Optional.empty`), sem expor o motivo da rejeição (RF06).
`PersistenceInfrastructureError`	`message: String`; `cause: Throwable`	Exceção técnica lançada pelo adaptador de persistência diante de indisponibilidade, recusa de conexão ou *timeout* na Base de Usuários (RF08). Seus atributos internos (`message` e `cause`) destinam-se apenas a diagnóstico interno e jamais são serializados para o cliente; o `AuthHttpController` converte a falha em resposta `HTTP 503` genérica.
`CryptographicVerificationError`	`message: String`; `cause: Throwable`	Exceção técnica interna lançada pelo adaptador criptográfico caso o hash armazenado/configurado esteja corrompido ou ocorra falha inesperada no motor Argon2id. Não é exposta ao cliente, sendo convertida pelo `AuthHttpController` em resposta `HTTP 500 Internal Server Error` genérica.

Fonte: Elaborada pelo autor.

Tabela 7 — Especificação Formal das Operações nas Portas da Arquitetura Hexagonal

Porta (Interface)	Operação	Parâmetros de Entrada	Retorno Esperado	Pré-condições e Pós-condições	Exceções e Condições de Erro
`AuthenticationUseCasePort`	`verifyCredentials`	`input: AuthCredentialsInput`	`AuthDecisionResult`	**Pré:** `identifier` e `password` validados sintaticamente.	Propaga `PersistenceInfrastructureError` (falha de acesso ao

Porta (Interface)	Operação	Parâmetros de Entrada	Retorno Esperado	Pré-condições e Pós-condições	Exceções e Condições de Erro
				Pós: Retorna a decisão de autenticação e, em caso positivo, a identidade pública do usuário (`userId` e `identifier`), sem manter estado de sessão entre requisições.	banco, CA05) ou `CryptographicVerificationError` (falha interna inesperada na verificação).
`UserRepositoryPort`	`findByIdentifier`	`identifier: String`	`Optional<UserRecord>`	**Pré:** `identifier` não vazio. **Pós:** Executa leitura sem alterar registros na base externa (CON01/PRE01), retornando `Optional.of(UserRecord)` se encontrado ou `Optional.empty` se inexistente.	Lança `PersistenceInfrastructureError` se ocorrer falha de comunicação, erro de driver ou *timeout* no SGBD (RF08).
`PasswordVerifierPort`	`verify`	`rawPassword: String`; `encodedHash: String`	`Boolean`	**Pré:** `encodedHash` em formato Argon2id válido (do registro real ou do hash fictício pré-configurado). **Pós:** Retorna `true` se a senha corresponder ao hash e `false` se divergir; a senha original não é persistida nem registrada em log.	Retorna `false` para senha incorreta (tratada como credencial inválida, `401`). Lança `CryptographicVerificationError` apenas se `encodedHash` estiver corrompido ou inválido (tratado como erro interno, `500`).

Fonte: Elaborada pelo autor.

Fluxos de Autenticação e Tratamento de Erros

O comportamento dinâmico dos componentes durante o processamento de uma requisição, modelado no Diagrama de Sequência UML (Figura 4), estrutura-se em três etapas sequenciais que cobrem os cinco critérios de aceitação principais (CA01 a CA05) e o tratamento de falhas internas inesperadas:

- **1. Etapa 1 — Recebimento e Validação Sintática na Borda HTTP (**`CA04`**):** A requisição `POST /auth/verify` atravessa o **Servidor Web / Reverse Proxy** e alcança o `AuthHttpController` (RF01). O controlador verifica imediatamente a validade estrutural do JSON e o preenchimento dos atributos `identifier` e `password` (RF02). Caso a entrada seja inválida (**CA04**), o fluxo é encerrado no próprio adaptador primário com o retorno de `HTTP 400 Bad Request` (`VALIDATION_ERROR`, RF07), impedindo qualquer invocação ao `AuthenticationService` ou ao banco de dados.
- **2. Etapa 2 — Consulta ao Repositório e Isolamento de Falha de Persistência (**`CA05`**):** Sendo a entrada válida, o controlador invoca `verifyCredentials` na porta `AuthenticationUseCasePort`. O `AuthenticationService` aciona `UserRepositoryPort.findByIdentifier`, executada pelo `DatabaseUserRepositoryAdapter` junto à **Base de Usuários** (RF03). Caso ocorra falha de conexão ou *timeout* (**CA05**), o adaptador intercepta a falha nativa do SGBD e lança `PersistenceInfrastructureError`, interrompendo o caso de uso antes da etapa de verificação de senha. O erro é propagado pelo serviço e convertido pelo `AuthHttpController` em uma resposta `HTTP 503 Service Unavailable` (`SERVICE_UNAVAILABLE`, RF08) genérica, sem serializar a exceção ou sua causa (`cause`), garantindo que falhas operacionais do banco não sejam confundidas com credenciais inválidas nem exponham detalhes internos de infraestrutura.
- **3. Etapa 3 — Verificação Criptográfica e Decisão de Autenticação (**`CA01`**,** `CA02`**,** `CA03` **e Erro Interno):** Concluída a leitura no repositório sem falha técnica, o `AuthenticationService` avalia o retorno:
  - **Caminho de Usuário Inexistente (**`CA02`**):** Quando `findByIdentifier` retorna `Optional.empty`, o `AuthenticationService` mitiga diferenças temporais observáveis (ADR01) acionando igualmente `PasswordVerifierPort.verify`, passando a senha recebida e um **hash Argon2id fictício válido, previamente configurado (**`DUMMY_HASH`**)**, de custo computacional equivalente aos hashes reais. O retorno booleano dessa verificação fictícia é ignorado, o serviço devolve `AuthDecisionResult(authenticated = false, userId = Optional.empty, identifier = Optional.empty)` e o controlador responde com `HTTP 401 Unauthorized` (`INVALID_CREDENTIALS`, RF06).

- **Caminho de Usuário Existente (`CA01` e `CA03`):** Quando o registro `UserRecord` é encontrado, o serviço aciona `PasswordVerifierPort.verify(password, userRecord.passwordHash)` (RF04), executada pelo `Argon2PasswordVerifierAdapter`. Se o retorno for `true` (**CA01**), o serviço emite `AuthDecisionResult(authenticated = true, userId = Optional.of(userRecord.id), identifier = Optional.of(userRecord.identifier))` e o controlador o mapeia para `HTTP 200 OK` com o objeto JSON `{ "status": "AUTHENTICATED", "user": { "id": "...", "identifier": "..." } }` (RF05). Se o retorno for `false` (**CA03**), o serviço emite `AuthDecisionResult(authenticated = false, userId = Optional.empty, identifier = Optional.empty)` e o controlador retorna `HTTP 401 Unauthorized` com exatamente o mesmo contrato JSON e código de status do cenário **CA02** (RF06).
- **Tratamento Excepcional de Falha Interna na Verificação:** Caso `PasswordVerifierPort.verify` lance a exceção `CryptographicVerificationError` (por exemplo, diante de um hash corrompido na base externa ou falha interna inesperada da biblioteca criptográfica), essa falha não é classificada como senha incorreta (`401`) nem como indisponibilidade do banco de dados (`503`). O erro é propagado até o `AuthHttpController`, que retorna uma resposta genérica `HTTP 500 Internal Server Error` sem divulgar detalhes técnicos ao cliente.
-

2.2.4. Decisões e Rastreabilidade

Decisões Arquiteturais e Requisitos de Qualidade

A Tabela 8 sintetiza as decisões arquiteturais (ADR01 a ADR04) adotadas para endereçar os atributos de qualidade (RNF01 a RNF04), explicitando os mecanismos técnicos incorporados aos componentes e as limitações conhecidas (*trade-offs*) de cada escolha (IEEE COMPUTER SOCIETY, 2024).

Tabela 8 — Síntese das Decisões Arquiteturais, Qualidades e Limitações Conhecidas

Decisão e Requisito	Mecanismo Arquitetural Adotado	Componentes Envolvidos	Trade-offs e Limitações Conhecidas
ADR01 — Segurança de Credenciais (*RNF01*)	Terminação TLS no Reverse Proxy; tráfego interno em rede isolada; verificação via Argon2id (`verify`); respostas `401` idênticas para CA02 e CA03; verificação	`Servidor Web`, `AuthenticationService`, `Argon2PasswordVerifierAdapter`, `AuthHttpController`	O uso de hash fictício no CA02 consome CPU deliberadamente em tentativas com usuários inexistentes e reduz diferenças temporais

Decisão e Requisito	Mecanismo Arquitetural Adotado	Componentes Envolvidos	Trade-offs e Limitações Conhecidas
	com hash fictício pré-configurado quando o usuário não existe; vedação de senhas e exceções internas em logs e respostas.		observáveis, embora não garanta tempos de resposta idênticos devido a variações de rede e SGBD.
ADR02 — Interoperabilidade HTTP/JSON (*RNF02*)	Exposição de endpoint `POST /auth/verify` com payloads JSON padronizados e mapeamento semântico para `200`, `400`, `401`, `503` (e `500` para falhas internas inesperadas) (MDN WEB DOCS, s.d.-b).	`AuthHttpController`	Restringe o protocolo de entrada a clientes compatíveis com HTTP/JSON, exigindo a criação de um novo adaptador primário caso outros protocolos sejam demandados no futuro.
ADR03 — Execução Stateless e Concorrência (*RNF03*)	Decisões projetadas para favorecer o comportamento operacional concorrente: processamento isolado sem retenção de estado de sessão entre requisições, controle de *timeout* na leitura do banco e conversão imediata de falhas.	`AuthenticationService`, `DatabaseUserRepository Adapter`	Como o serviço não mantém cache de sessão nem emite tokens (CON01), toda requisição de verificação exige consulta ao repositório e cálculo do Argon2id. O atendimento efetivo às metas de desempenho, latência e capacidade depende de verificação prática por testes de carga e medições.
ADR04 — Desacoplamento Hexagonal (*RNF04*)	Inversão de dependências por meio das portas `AuthenticationUseCaseP ort`, `UserRepositoryPort` e `PasswordVerifierPort` (COCKBURN, 2005).	Todos os componentes da `Aplicação de Autenticação`	Introduz interfaces e abstrações adicionais em comparação a uma organização com dependências diretas entre componentes, em troca de maior isolamento tecnológico, testabilidade e facilidade de manutenção.

Fonte: Elaborada pelo autor com base na Especificação de Requisitos (Subseção 2.1).

Matriz de Rastreabilidade

A Tabela 9 consolida as relações entre os requisitos funcionais (RF01 a RF08), os requisitos não funcionais e decisões arquiteturais (RNF01/ADR01 a RNF04/ADR04), os critérios de aceitação (CA01 a CA05), os componentes responsáveis e os artefatos de modelagem do repositório (IEEE COMPUTER SOCIETY, 2024).

Tabela 9 — Matriz de Rastreabilidade Arquitetural

Requisito	Descrição Sintética	Componentes Responsáveis	Critérios de Aceitação	Decisões e RNFs Relacionados	Artefatos de Modelagem Associados
RF01	Receber credenciais via requisição HTTP	`Servidor Web`, `AuthHttpController`	CA01 a CA05	RNF01/ADR01, RNF02/ADR02, CON02	`contexto.puml`, `conteineres.puml`, `componentes.puml`, `sequencia-autenticacao.puml`
RF02	Validar sintaxe e campos obrigatórios de entrada	`AuthHttpController`	CA04	RNF02/ADR02	`componentes.puml`, `sequencia-autenticacao.puml`
RF03	Consultar existência do usuário no repositório	`AuthenticationService`, `UserRepositoryPort`, `DatabaseUserRepositoryAdapter`	CA01, CA02, CA03, CA05	RNF03/ADR03, RNF04/ADR04, CON01, PRE01	`contexto.puml`, `conteineres.puml`, `componentes.puml`, `sequencia-autenticacao.puml`
RF04	Verificar senha utilizando o hash armazenado (ou fictício)	`AuthenticationService`, `PasswordVerifierPort`, `Argon2PasswordVerifierAdapter`	CA01, CA02, CA03	RNF01/ADR01, RNF04/ADR04	`componentes.puml`, `sequencia-autenticacao.puml`
RF05	Retornar sucesso com a identificação do usuário	`AuthenticationService` (decisão), `AuthHttpController` (HTTP 200)	CA01	RNF02/ADR02	`contexto.puml`, `componentes.puml`, `sequencia-autenticacao.puml`
RF06	Retornar erro genérico para credenciais inválidas	`AuthenticationService` (rejeição), `AuthHttpController` (HTTP 401)	CA02, CA03	RNF01/ADR01, RNF02/ADR02	`contexto.puml`, `componentes.puml`, `sequencia-autenticacao.puml`
RF07	Retornar erro de validação para entrada inválida	`AuthHttpController` (HTTP 400)	CA04	RNF02/ADR02	`componentes.puml`, `sequencia-autenticacao.puml`
RF08	Tratar e retornar erro técnico de falha de infraestrutura	`DatabaseUserRepositoryAdapter`, `AuthenticationService`,	CA05	RNF02/ADR02, RNF03/ADR03	`componentes.puml`, `sequencia-autenticacao.puml`

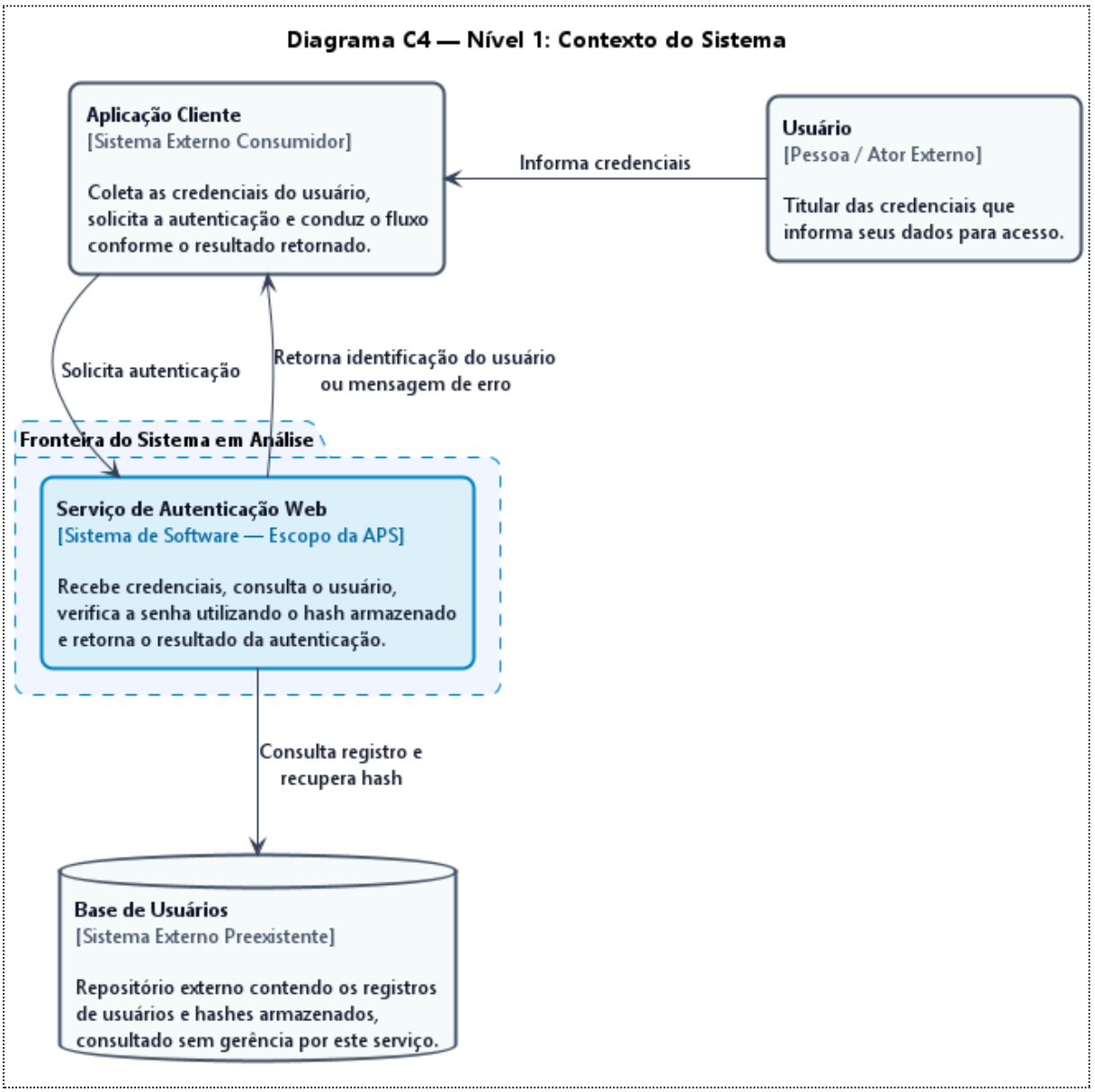
Requisito	Descrição Sintética	Componentes Responsáveis	Critérios de Aceitação	Decisões e RNFs Relacionados	Artefatos de Modelagem Associados
		`AuthHttpController` (HTTP 503)			

Fonte: Elaborada pelo autor.

2.3. Diagramas Arquiteturais e Comportamentais (C4 e UML)

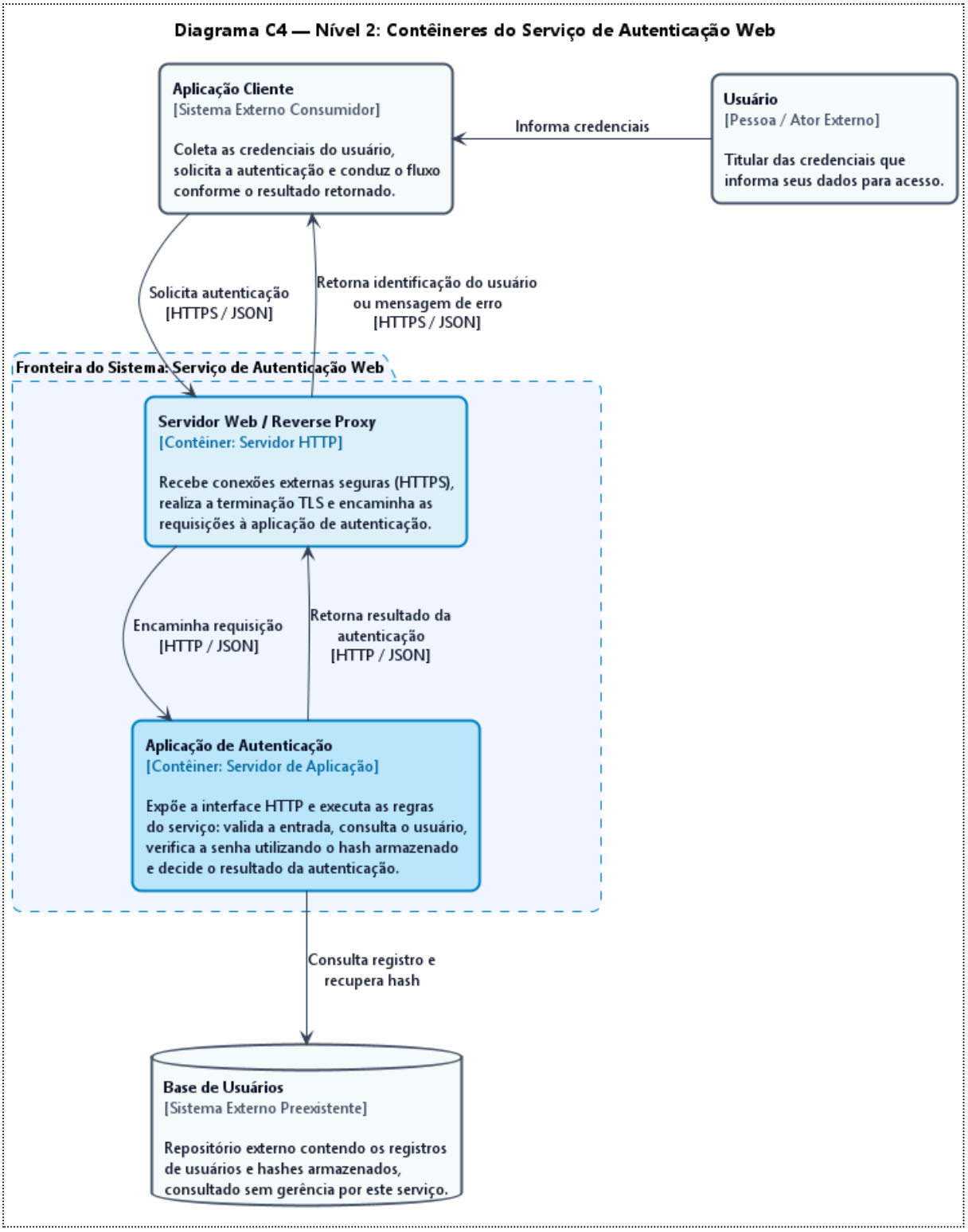
Nesta subseção apresentam-se os quatro diagramas oficiais que formalizam as visões estática e dinâmica do **Serviço de Autenticação Web**, modelados segundo a notação **C4 Model** (BROWN, 2018) e a linguagem **UML**, em correspondência direta com os requisitos (RF01 a RF08), critérios de aceitação (CA01 a CA05) e componentes da Arquitetura Hexagonal especificados nas subseções anteriores.

Figura 1 — Diagrama C4 — Nível 1: Contexto do Sistema



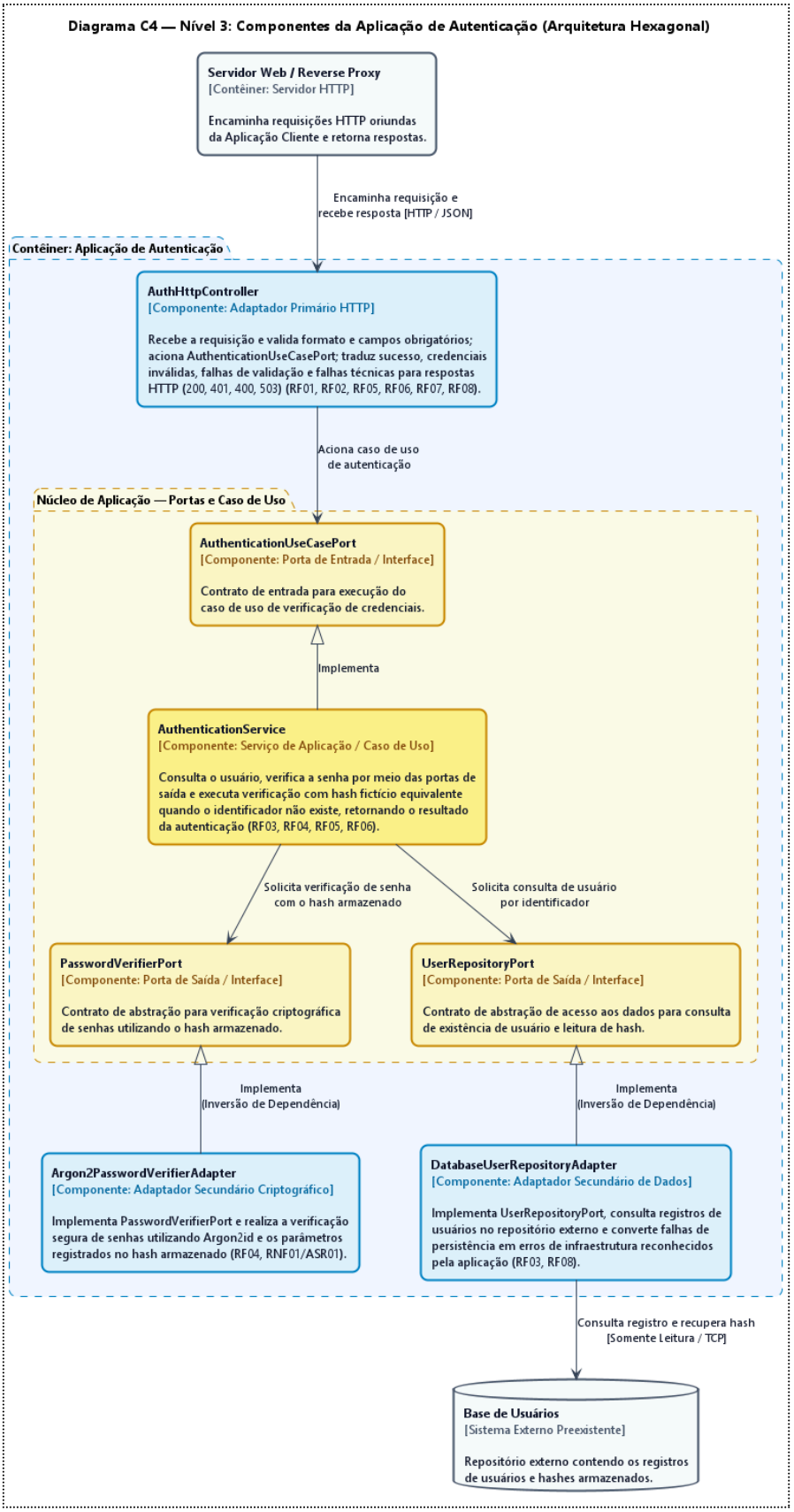
Fonte: Elaborada pelo autor a partir do artefato oficial diagramas/c4/contexto.puml.

Figura 2 — Diagrama C4 — Nível 2: Contêineres do Serviço de Autenticação Web



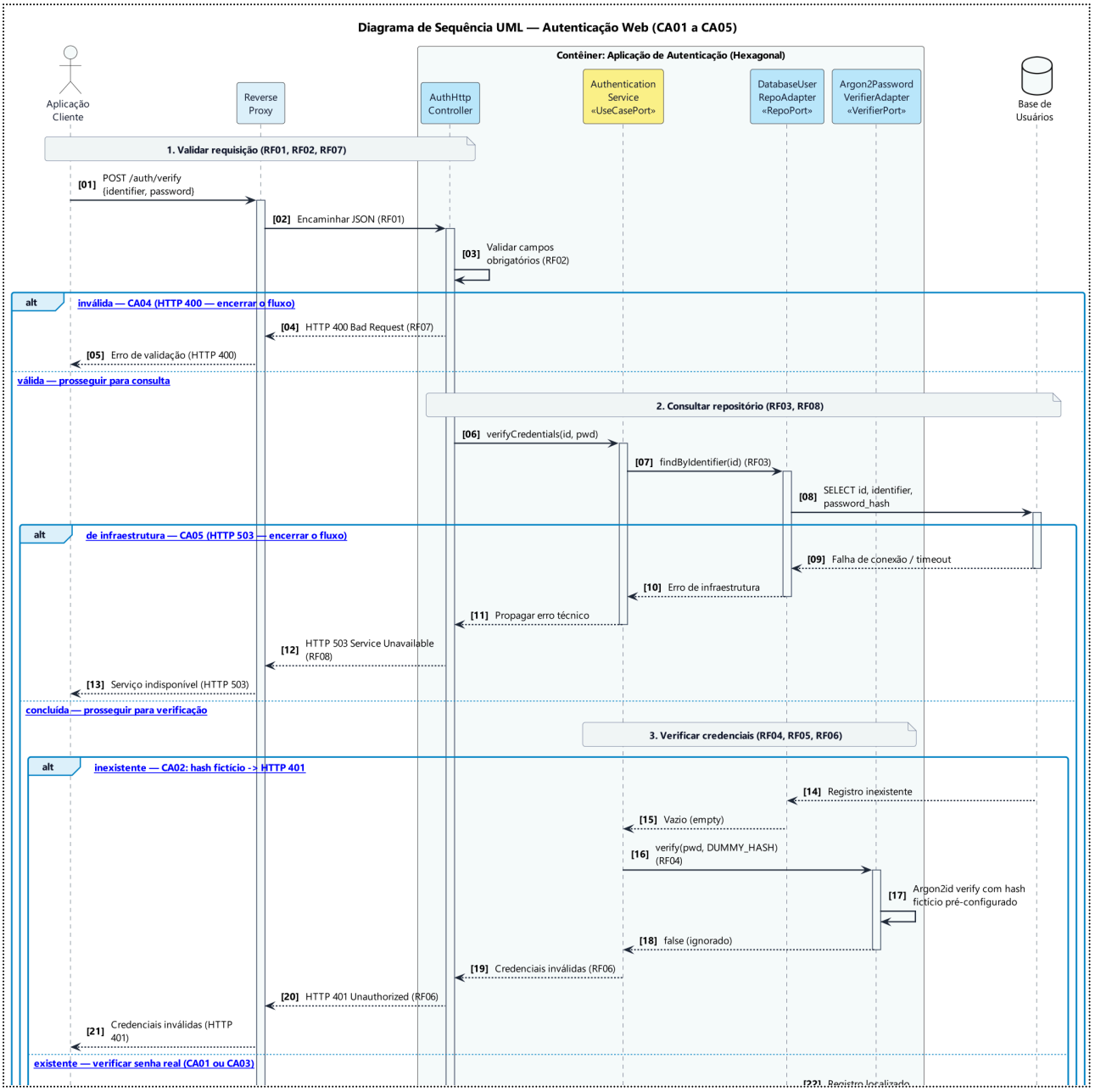
Fonte: Elaborada pelo autor a partir do artefato oficial diagramas/c4/conteineres.puml.

Figura 3 — Diagrama C4 — Nível 3: Componentes da Aplicação de Autenticação (Arquitetura Hexagonal)



Fonte: Elaborada pelo autor a partir do artefato oficial diagramas/c4/componentes.puml.

Figura 4 — Diagrama de Sequência UML — Autenticação Web (CA01 a CA05)



Fonte: Elaborada pelo autor a partir do artefato oficial diagramas/uml/sequencia-autenticacao.puml.

Referências Bibliográficas

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. **Software Architecture in Practice**. 4. ed. Boston: Addison-Wesley, 2021.

BROWN, Simon. **The C4 model for visualising software architecture**. 2018. Disponível em: https://c4model.com/. Acesso em: 12 nov. 2026.

COCKBURN, Alistair. **Hexagonal architecture**. 2005. Disponível em: https://alistair.cockburn.us/hexagonal-architecture/. Acesso em: 12 nov. 2026.

FIELDING, Roy T.; TAYLOR, Richard N. Principled design of the modern Web architecture. **ACM Transactions on Internet Technology (TOIT)**, New York, v. 2, n. 2, p. 115-150, maio 2002.
IEEE COMPUTER SOCIETY. **Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)**. Version 4.0. Editado por Hironori Washizaki. IEEE Computer Society, 2024. Disponível em: https://www.swebok.org/. Acesso em: 09 nov. 2026.
INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 12207: Systems and software engineering — Software life cycle processes**. Geneva: ISO, 2017.
INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 29148: Systems and software engineering — Life cycle processes — Requirements engineering**. Geneva: ISO, 2018.
INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 42010: Systems and software engineering — Architecture description**. Geneva: ISO, 2011.
KRUCHTEN, Philippe. Architectural blueprints—the "4+1" view model of software architecture. **IEEE Software**, v. 12, n. 6, p. 42-50, nov. 1995.
MDN WEB DOCS. **Como a Web funciona**. [s.d.-a]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works. Acesso em: 10 nov. 2026.
MDN WEB DOCS. **Uma visão geral do HTTP**. [s.d.-b]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Guides/Overview. Acesso em: 10 nov. 2026.
OWASP FOUNDATION. **Password Storage Cheat Sheet**. [s.d.]. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 12 nov. 2026.
OPENAI. ChatGPT. Assistente de inteligência artificial. Disponível em: https://chatgpt.com/. Acesso em: 12 nov. 2026. Questões: Apêndice II, Seção II.2.

Apêndice I — Registro de Utilização de Inteligência Artificial

1. Ferramenta Utilizada

ChatGPT (OpenAI).

2. Finalidade da Utilização

Utilizei a ferramenta como um apoio técnico e metodológico durante o desenvolvimento do projeto, principalmente para organizar a documentação, revisar os textos e estruturar os diagramas arquiteturais. No início, a IA me ajudou a definir uma divisão lógica para os arquivos, títulos e subtítulos, garantindo que tudo seguisse o escopo da atividade. Toda a redação dos documentos foi conduzida por mim, mas contei com revisões periódicas da ferramenta para identificar possíveis inconsistências e pontos de melhoria.

3. Atividades com Apoio de IA

O suporte da inteligência artificial concentrou-se na revisão crítica dos textos, na conferência dos requisitos e no desenvolvimento dos artefatos de modelagem, sempre alinhados aos conceitos estudados em Domínios de Software.
Na criação dos diagramas C4 e UML, essa colaboração foi mais próxima. O processo funcionou como uma troca: eu apresentava as propostas iniciais, definia os elementos e discutia os relacionamentos necessários, enquanto a IA ajudava a estruturar os modelos, conectar os componentes e resolver cenários mais complexos. Em alguns momentos, a ferramenta chegou a sugerir e montar trechos da modelagem, que eu analisava e ajustava em seguida.
Além disso, a IA foi útil para apontar erros de sintaxe, lacunas técnicas, falhas arquiteturais e eventuais divergências entre os diagramas e o texto. Esse acompanhamento contínuo permitiu refinar os materiais aos poucos, assegurando que a solução proposta estivesse bem alinhada aos requisitos.

4. Validação e Responsabilidade Autoral

Durante todo o trabalho, a ferramenta atuou na tutoria, na revisão técnica e no suporte à modelagem, facilitando tanto a organização das etapas quanto o aprimoramento das entregas.
Mesmo com a participação direta da IA na montagem de partes dos diagramas, a definição do escopo, a análise das propostas, a escolha das soluções e a validação final ficaram totalmente sob minha responsabilidade. Todas as sugestões geradas pela ferramenta passaram por uma avaliação criteriosa e pelos ajustes necessários para garantir que atendessem aos objetivos da atividade.
Assim, a inteligência artificial funcionou como uma ferramenta de apoio no desenvolvimento do projeto, mantendo comigo o controle sobre as decisões técnicas e a qualidade final dos artefatos.

Apêndice II — Registro dos Prompts de Inteligência Artificial

II.1. Identificação da Ferramenta

- **Ferramenta:** ChatGPT
- **Organização:** OpenAI
- **Endereço eletrônico:** https://chatgpt.com/
- **Finalidade:** Apoio metodológico, revisão técnica, verificação de consistência e análise dos artefatos da APS.

II.2. Prompts Utilizados

Este apêndice apresenta o registro dos prompts utilizados durante o desenvolvimento da APS, contemplando as interações que conduziram a estruturação da documentação, a especificação dos requisitos, a modelagem arquitetural e a revisão técnica dos artefatos.

II.2.1. Organização da Documentação

Artefatos relacionados: Estrutura documental e organização do repositório.
Prompt original:

> *Olá, quero que analise muito bem a fundo e com extrema atenção o que o trabalho pede. Preciso realizar a documentação de algo relacionado a Web e uma documentação inteira, nisso o trabalho deverá ser armazenado no GitHub além de*

commites serem frequentes. Me informe o que devo fazer passo a passo. Eu disponibilizei também o capítulo do Swebook que fala sobre a arquitetura, acredito que pode ajudar.

Contribuição obtida: Orientações para organizar a documentação conforme as partes exigidas na APS.

Quero que analise esses 2 docs e leia os capitulos 6.4.2 e 6.4.3 da norma 15288. Após isso, avalie como elas se relacionam com o trabalho feito até agora. Issó é pelo fato de que eu refiz o nosso documento adaptando-o conforme o Swebok-v4 e a norma ISO 15288. O novo documento que remodelei está em formato .docx

II.2.2. Especificação de Requisitos

Artefato relacionado: `especificacao-requisitos.md`

Prompt original:

Certo, vamos começar os proximos passos, se forem os requisitos, o cuidado deve ser redobrado e seguir uma extrutura muito bem estabelecida. Eu já possuo um documento extruturado para isso, mas ele não está 100% completo, então preciso que você me ajude a complementá-lo.

Você não acha que esses requisitos estão grandes e detalhados demais? Tenho a impressão que ocorreu uma fusão nos requisitos, geralmente eu costumo vê-los mais fragmentados, mas posso estar 100% enganado.

Contribuição obtida: Identificação de inconsistências e sugestões de melhoria na especificação.

II.2.3. Esboço e desenvolvimento dos diagramas

Artefatos relacionados: `sequencia-autenticacao.puml` e `conteineres.puml`

Prompt original:

Para o próximo diagrama, vamos detalhar como o sistema de autenticação funciona por dentro, que é nível de Containers. Precisamos incluir os seguintes elementos: Servidor Web/Reverse Proxy: vai receber as conexões Https e repassar as requisições para a aplicação, seguindo o que pede a restrição CON02; Aplicação de Autenticação: onde rodará as regras de negócio e que expõe a interface HTTP; Base de Usuários externa: continua de fora da nossa fronteira arquitetural por enquanto; Aplicação Cliente: segue como um sistema externo que consome o serviço.

Sobre o escopo dos fragmentos alt (prioridade alta): este é o ponto mais importante para ajustarmos antes de finalizar esse diagrama, pois notei um problema na estrutura. Na primeira etapa, você define que uma entrada inválida retorna HTTP 400 e uma entrada válida prossegue, mas como o fragmento alt fecha antes da etapa 2, o diagrama não garante formalmente que a consulta ao repositório vai acontecer apenas se a entrada for válida. O mesmo problema ocorre no cenário CA05, onde a etapa de verificação criptográfica ficou fora do fragmento alternativo após o retorno do HTTP 503, o que me passa impressão de que o sistema continua processando a requisição mesmo depois de uma falha na infraestrutura.

Contribuição obtida: Correção técnica sobre a modelagem arquitetural.

II.2.4. Verificação Final dos Artefatos

Artefatos relacionados: Documentação, referências e diagramas.

Prompt original:

Certo, o diagrama C4 de containeres foi corrigido conforme você pediu. Agora temos o de componentes, peço que analise esse com mais cuidado, está mais complexo.

Contribuição obtida: Conferência de rastreabilidade, referências, links e coerência entre os artefatos.

Aqui estão as referencias, eles tiveram alterações.